Universidad de Costa Rica

Facultad de Física-Matemática
Escuela de Matemática

OPTIMIZACIÓN DE RUTAS DE ENTREGA

Proyecto Grupal

Estudiantes:
Paola Espinoza Hernández, C32715
Gabriel Sanabria Alvarado, C27184

Curso:
Herramientas para Ciencias de Datos II

Profesor:
Luis Alberto Potoy Juarez

Fecha de entrega:
Junio 2025

San José, Costa Rica
2025

Repositorio: Optimización de rutas de entrega
Darshboard interactivo: click aquí

1. Introducción

El crecimiento del comercio electrónico y los servicios logísticos ha provocado que la optimización de rutas de entrega se vuelva un factor fundamental para empresas de transporte y distribución. La eficiencia en el uso de vehículos, la reducción de costos operativos y la mejora en los tiempos de entrega inciden directamente en la rentabilidad y satisfacción del cliente.

Este proyecto tiene como objetivo desarrollar una solución computacional en Python que permita encontrar rutas de entrega óptimas para un conjunto de ubicaciones dentro de un entorno urbano. Se pretende enfatizar la minimización de las distancias recorridas por los vehículos, tomando en cuenta los costos de transporte.

Para alcanzar este objetivo, se utilizan herramientas de análisis geoespacial como *OSMnx*, además de algoritmos de optimización del módulo OR-Tools de Google, usualmente utilizado para resolver problemas clásicos como el Problema del Viajante (TSP) o el Problema de Ruteo de Vehículos (VRP) [Google, 2023].

Este enfoque no solo ofrece una solución técnica eficiente, sino que también proporciona una plataforma extensible para analizar escenarios reales, tomar decisiones basadas en datos y mejorar la sostenibilidad del transporte urbano.

2. Metodología

Tras la exploración del estado de las herramientas actuales, se definió una metodología basada en programación en *Python*, apoyada por bibliotecas como **OSMnx**, **NetworkX** y **OR-Tools**, con el objetivo de construir una solución robusta y escalable para la optimización de rutas de entrega en entornos urbanos. El proceso inicia con una lista de lugares definidos por el usuario. Mediante la función `geocode()` de **OSMnx**, dichas ubicaciones se traducen a coordenadas geográficas (latitud y longitud), lo cual permite representar espacialmente los puntos de entrega sobre el mapa urbano real.

Con las coordenadas obtenidas, se genera un grafo de calles mediante el método `graph_from_bbox()`, también proporcionado por **OSMnx**. Este grafo representa la red vial real, donde los nodos corresponden a intersecciones y las aristas a segmentos de calles.

La generación del grafo se limita a una zona geográfica específica, definida mediante un *bounding box*, que es un rectángulo que encierra todos los puntos seleccionados; utilizado como marco de referencia para descargar solo las calles que se encuentran dentro del área de interés.

Para realizar una estimación de las distancias, se utiliza la librería **NetworkX**, con esto se determinan las distancias más cortas entre cada par de puntos. Para ello se emplea la función `nx.shortest_path_length()`, que utiliza el algoritmo de Dijkstra, el cual toma en cuenta la longitud real de las calles (almacenada como atributo `length` en las aristas) y construye una matriz de distancias expresadas en kilómetros.

En términos prácticos, se parte de una red vial obtenida mediante *OSMnx*, donde cada intersección se representa como un nodo y cada calle como una arista con un peso correspondiente a su longitud. Luego, para una lista de coordenadas geográficas, se identifica el nodo más cercano mediante la función `ox.nearest_nodes()`, y se calcula la distancia más corta entre pares con `nx.shortest_path_length(G, origen, destino, weight='length')`, dividiendo entre 1000 para obtener kilómetros.

Con la matriz de distancias como entrada, se formula un problema del Viajante de Comercio (TSP) utilizando el módulo **OR-Tools** de Google. A través del `Routing Solver`, se construye un modelo con un único vehículo que parte de un nodo base y debe visitar todos los puntos exactamente una vez, minimizando la distancia total recorrida.

La estrategia utilizada es *Path Cheapest Arc*, que construye una solución inicial de manera rápida: comienza en el nodo de origen y, en cada paso, se dirige al nodo más cercano que aún no ha sido visitado. Es decir, siempre elige la conexión disponible con menor distancia. Aunque esta estrategia no garantiza la ruta óptima, permite obtener rápidamente una solución razonablemente buena, que luego puede ser mejorada mediante técnicas adicionales de optimización [Google OR-Tools Team, 2024].

Para mostrar los resultados, el sistema genera la mejor secuencia de visitas, calcula el recorrido total en kilómetros y ofrece dos formas de ver la ruta:

- **Visualización estática con grafos:** Usando la biblioteca `OSMnx` y la clase `Graficar`, se crea una imagen del mapa vial donde se resalta la ruta óptima. Esta vista es útil para analizar con detalle las conexiones entre los puntos y el recorrido completo.

- **Visualización interactiva en mapas web:** Con `Folium`, se hacen mapas que permiten explorar la ruta de forma dinámica, mostrando los puntos de visita y la ruta real sobre el mapa. Se pueden activar o desactivar partes específicas para un análisis más detallado.

Con estos métodos se permite integrar datos del mundo real con herramientas de optimización para generar soluciones aplicables a contextos urbanos reales, tal como se ha propuesto en investigaciones recientes ([Muñoz et al., 2024, Google, 2023, Castro, 2022, Boeing, 2025, Hagberg et al., 2008]).

La solución desarrollada para este proyecto se construye en Python utilizando tres clases principales: `Mapa`, `Distancias` y `Enrutamiento`. Cada una de estas clases tiene una tarea específica y trabaja de forma independiente, lo que facilita organizar el código,

entenderlo mejor y hacer cambios sin afectar todo el sistema. Para ello, se apoya en bibliotecas especializadas como **OSMnx**, **NetworkX**, **NumPy** y **OR-Tools**.

La clase `Mapa` representa el primer paso del proceso. Recibe como entrada una lista de ubicaciones proporcionadas por el usuario en forma de nombres de lugares. Utilizando la función `osmnx.geocode()` de la biblioteca OSMnx, transforma estas ubicaciones en coordenadas geográficas (latitud y longitud). Posteriormente, construye un grafo de calles mediante la función `graph_from_bbox()`, que descarga la red vial contenida dentro de un rectángulo geográfico que engloba todos los puntos indicados. Este grafo representa la infraestructura vial extraída de **OpenStreetMap** y se configura específicamente para el tráfico vehicular usando el parámetro `network_type='drive'`.

En cuanto a la clase `Distancias`, esta es la encarga de calcular la matriz de distancias mínimas entre todos los pares de ubicaciones. Para esto, identifica los nodos del grafo más cercanos a cada punto de interés utilizando `ox.nearest_nodes()` y luego aplica el algoritmo de caminos más cortos (`nx.shortest_path_length()` de NetworkX) ponderado por la longitud de las calles. Así se obtiene una matriz cuadrada donde cada elemento representa la distancia real en carretera (en kilómetros) entre dos ubicaciones, reflejando trayectos viables y no simplemente distancias rectas.

Una vez obtenida la matriz de distancias, la clase `Enrutamiento` se encarga de resolver el problema del viajante (TSP) a través de la biblioteca OR-Tools de Google. Define un modelo con un solo vehículo, un punto de inicio (denominado depósito) y una función de costo, por defecto, distancia. Utiliza la estrategia `PATH_CHEAPEST_ARC` para encontrar la ruta más económica. El resultado es una ruta optimizada que recorre todos los puntos una única vez, minimizando la distancia total del trayecto. Esta ruta se presenta como una lista de la secuencia de nodos, acompañada del valor objetivo (distancia total) para su fácil interpretación.

Además, se desarrolló una implementación visual para representar el grafo vial, los puntos de interés y las rutas óptimas utilizando la biblioteca `OSMnx`, complementada con visualizaciones mediante `matplotlib`. A través de funciones como `ox.plot_graph_route()`, se logró graficar el camino más corto entre puntos de entrega sobre el grafo vial extraído de datos de OpenStreetMap, facilitando el análisis visual y la interpretación de los resultados.

Además, se implementó una clase llamada `MapaInteractivo`, cuyo propósito es generar visualizaciones interactivas a partir de los resultados obtenidos. Esta clase recibe como entrada el grafo vial generado, las coordenadas geográficas de los puntos de interés y la secuencia de visitas producida por el modelo de optimización. Luego, utilizando `Folium`, se construye un mapa que muestra los marcadores correspondientes a cada ubicación, con íconos y etiquetas personalizadas, y traza la ruta óptima como una polilínea sobre la red vial [Developers, 2024]. La función `crear_mapa_interactivo()` permite dividir los tramos del recorrido en capas activables individualmente, con lo cual se puede explorar

visualmente cada segmento del trayecto. El resultado final se puede exportar como archivo HTML, lo cual facilita su visualización en navegadores web sin necesidad de herramientas externas.

3. Resultados

El sistema desarrollado fue aplicado a un conjunto de ubicaciones urbanas localizadas en el área metropolitana de San José, Costa Rica. Las localidades seleccionadas fueron Tibás, Escazú, Curridabat y San Pedro. Estas ubicaciones se ingresaron como una lista de nombres de lugares, los cuales fueron transformados en coordenadas geográficas (latitud y longitud) utilizando la función `geocode()` de la biblioteca `OSMnx`. Este paso permitió establecer una representación espacial precisa de los puntos de interés sobre una red vial real.

Una vez obtenidas las coordenadas, se generó un grafo vial a partir de la función `graph_from_bbox()`, también de `OSMnx`. Este grafo incluyó únicamente los nodos y aristas correspondientes a calles contenidas dentro de un rectángulo geográfico mínimo que abarcaba las cuatro ubicaciones. Cada nodo del grafo representa una intersección vial, y cada arista corresponde a un segmento de calle, con un atributo asociado de longitud en metros.

Posteriormente, cada una de las coordenadas fue asignada al nodo más cercano en la red vial utilizando la función `ox.nearest_nodes()`. Con estos nodos de referencia, se calculó la matriz de distancias mínimas entre cada par de ubicaciones mediante la función `nx.shortest_path_length()` de la biblioteca `NetworkX`, especificando el atributo `length` como peso. Esto permitió obtener las distancias reales sobre la red vial, expresadas en kilómetros, al dividir el resultado entre 1000.

Con esta matriz de distancias como insumo, se formuló el problema del viajante (TSP) usando la herramienta `Routing Solver` de `OR-Tools`. El modelo implementado consistió en un solo vehículo con punto de inicio fijo, configurado para visitar exactamente una vez cada nodo y retornar al origen. La estrategia de solución empleada fue `PATH_CHEAPEST_ARC`, la cual construye una ruta conectando el nodo actual con el siguiente nodo disponible que tenga el menor costo según la matriz de distancias. Esta configuración produjo una ruta cuya secuencia de visitas fue: Tibás → Escazú → San Pedro → Curridabat → Tibás. La distancia total del recorrido fue de aproximadamente **34.7 km**.

Para verificar el correcto funcionamiento del sistema, se realizó una representación gráfica de cada uno de los tramos obtenidos en la ruta. Estas visualizaciones se generaron mediante la función `ox.plot_graph_route()`, la cual permite trazar sobre el grafo vial los caminos más cortos entre cada par de nodos especificados. A continuación, se muestran las representaciones gráficas de los cuatro tramos consecutivos que conforman el recorrido

completo:

Figura 1: Tibás a Escazú

Figura 2: Escazú a San Pedro

Figura 3: San Pedro a Curridabat

Figura 4: Curridabat a Tibás

Cada una de estas imágenes refleja el trazado del recorrido óptimo sobre el grafo vial extraído de OpenStreetMap. El camino entre nodos fue determinado por el algoritmo de Dijkstra, tomando en cuenta la longitud real de las calles. Se validó visualmente que las rutas generadas corresponden a trayectos viables dentro de la infraestructura vial de San José.

Además de las representaciones estáticas, se generó un mapa interactivo con la biblioteca `Folium`, que permite la visualización dinámica de la ruta optimizada. Este mapa fue construido a partir de las coordenadas geográficas de los puntos de interés, junto con

los tramos calculados mediante la matriz de distancias. El resultado se integró en un archivo HTML interactivo donde se puede observar la secuencia completa del recorrido, activar o desactivar tramos específicos y visualizar los nombres de los lugares. La figura a continuación muestra una captura del mapa generado:

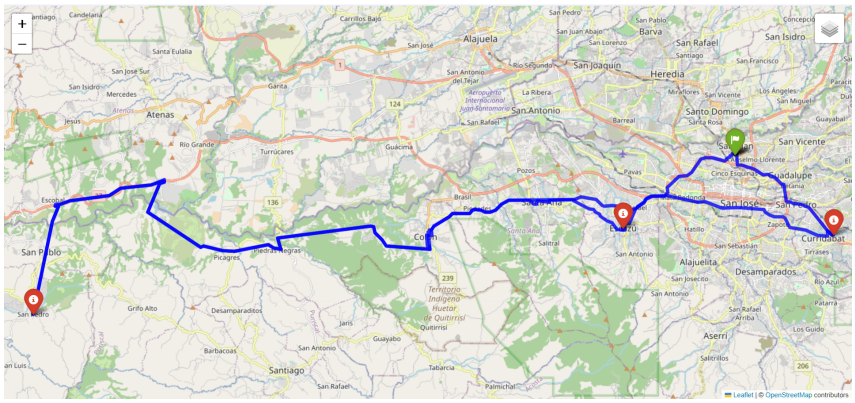

Figura 5: Visualización del mapa interactivo generado con Folium. Abrir HTML

Los resultados obtenidos cumplen con las condiciones del problema del viajante clásico: cada nodo fue visitado una sola vez y el recorrido retornó al punto de partida. La solución generada corresponde a una secuencia cerrada de visitas, optimizada en función de la distancia total. Se verificó que las ubicaciones fueron correctamente mapeadas a la red vial, y que las rutas entre ellas siguen caminos factibles dentro del grafo.

El comportamiento del sistema fue consistente con las expectativas planteadas. Las herramientas utilizadas (OSMnx, NetworkX, OR-Tools, Folium) funcionaron de forma integrada, y la implementación desarrollada produjo resultados coherentes y reproducibles. Las imágenes estáticas permitieron revisar la estructura del grafo y los trayectos calculados, mientras que la visualización interactiva facilitó la revisión completa de la ruta en un entorno navegable.

- Todas las ubicaciones fueron conectadas mediante rutas viables sobre la red vial, sin inconsistencias en la generación de caminos entre pares de nodos.
- El modelo de optimización retornó una única solución con la secuencia de visitas esperada y un recorrido cerrado.
- La distancia total del recorrido, de aproximadamente 34.7 km, coincide con el valor calculado por la suma de distancias de caminos más cortos entre puntos consecutivos.
- No se identificaron errores de ejecución, pérdidas de nodos, ni rutas no conectadas en ninguna de las fases del procesamiento.

Con base en estos resultados, se confirma que el sistema es capaz de resolver instancias del problema del viajante sobre mapas urbanos reales, empleando datos geográficos actuales y algoritmos de optimización. Las salidas generadas permiten interpretar y validar

el proceso completo desde la entrada de ubicaciones hasta la obtención de trayectorias viables.

4. Conclusiones

El desarrollo de este proyecto permitió demostrar que es posible construir un sistema funcional para la optimización de rutas de entrega en entornos urbanos, utilizando exclusivamente herramientas de código abierto y datos geoespaciales públicos. A partir de una lista de ubicaciones, el sistema genera coordenadas geográficas mediante geocodificación, construye una red vial real extraída de OpenStreetMap, y calcula las distancias mínimas entre puntos usando el algoritmo de Dijkstra sobre un grafo orientado y ponderado.

La formulación del problema se realizó como una instancia clásica del problema del viajante (TSP), resuelto con la biblioteca `OR-Tools` mediante la estrategia `Path Cheapest Arc`. Esta configuración permitió generar rutas cerradas que minimizan la distancia total, cumpliendo con la condición de visitar cada ubicación exactamente una vez y retornar al punto de inicio. La solución obtenida fue coherente con la infraestructura vial de la zona metropolitana de San José, Costa Rica.

La implementación del sistema fue estructurada en clases independientes para el manejo del grafo vial, el cálculo de distancias y la resolución del modelo de rutas, lo cual facilitó la organización del código y su mantenimiento. Además de las visualizaciones estáticas generadas con `OSMnx`, se implementó una clase adicional (`MapaInteractivo`) que construye mapas dinámicos con `Folium`, permitiendo representar los tramos del recorrido y los puntos de entrega sobre un mapa web navegable.

Los resultados muestran que el sistema es capaz de generar trayectorias viables con tiempos de procesamiento bajos, utilizando como insumo únicamente los nombres de las ubicaciones. Las herramientas empleadas funcionaron de forma integrada y el sistema respondió correctamente ante la estructura de datos y el grafo vial generado.

El enfoque adoptado es compatible con futuras extensiones. Si bien el modelo actual considera un único vehículo sin restricciones operativas, puede ser ampliado para incorporar condiciones como ventanas de tiempo, capacidades vehiculares o múltiples rutas. También sería posible integrar datos adicionales como flujos de tráfico, prioridades de entrega u horarios de operación para abordar variantes más complejas del problema original.

En conjunto, el sistema desarrollado resuelve un problema concreto de planificación logística urbana, y permite explorar nuevas aplicaciones relacionadas con el análisis espacial, la gestión de transporte y la optimización de recursos en entornos urbanos.

5. Recomendaciones

Es importante que el conjunto de lugares ingresado al sistema esté correctamente especificado, utilizando nombres geográficos que puedan ser reconocidos por el servicio de geocodificación proporcionado por la biblioteca `OSMnx`. Se recomienda realizar una verificación manual de la correspondencia entre los nombres ingresados y las ubicaciones geográficas reales, especialmente en casos donde los nombres puedan ser ambiguos, repetidos o susceptibles de interpretaciones múltiples por parte del servicio de geocodificación.

Durante la construcción del grafo vial, se aconseja no utilizar más de 20 ubicaciones en una misma ejecución. Esto se debe a que la descarga y el procesamiento del grafo mediante la función `graph_from_bbox()` puede implicar una carga computacional significativa, particularmente si las ubicaciones se encuentran dispersas geográficamente. El área de análisis se define automáticamente a partir del rectángulo mínimo que contiene todas las coordenadas geográficas (*bounding box*), por lo que se recomienda que los puntos estén relativamente próximos entre sí. Esto evita la descarga excesiva de datos viales que podrían no ser relevantes para la resolución del problema.

En el cálculo de la matriz de distancias mínimas entre puntos, es importante tener en cuenta que, si no existe una ruta viable entre dos ubicaciones debido a interrupciones en la red vial o a deficiencias en los datos de OpenStreetMap, el sistema asignará un valor nulo en la celda correspondiente. Por ello, es fundamental interpretar cuidadosamente los resultados obtenidos y revisar con atención cualquier valor cero o nulo que aparezca en la matriz, especialmente si ocurre en contextos en los que se esperaría una conexión directa.

Respecto al proceso de enrutamiento, la clase `Enrutamiento` está diseñada para trabajar bajo el supuesto de un único vehículo y una matriz de distancias previamente construida. El nodo correspondiente al punto de inicio, denominado depósito, debe ser definido de forma precisa para que el algoritmo pueda generar correctamente una ruta circular. Si bien el sistema utiliza distancias físicas como función de costo por defecto, es posible modificar la matriz de entrada para emplear otros criterios, siempre que se mantenga la estructura esperada por el solucionador de `OR-Tools`.

Para la visualización de los resultados, se recomienda ejecutar las funciones en un entorno con soporte gráfico, como `Jupyter Notebook`, `Google Colab` o entornos de desarrollo local con visualización habilitada. Las visualizaciones generadas pueden exportarse en formatos estáticos como PNG o dinámicos como HTML, lo cual permite su revisión posterior, integración en informes o distribución a terceros.

Adicionalmente, se sugiere incorporar un mecanismo de validación de entradas por parte del usuario, que permita verificar que los nombres de ubicaciones o coordenadas proporcionadas sean compatibles con el sistema de geocodificación. Esto puede prevenir errores de ejecución y mejorar la confiabilidad del sistema. También sería conveniente permitir la personalización del área geográfica de análisis mediante parámetros ajustables

que controlen los márgenes del *bounding box*. De esta forma, el usuario podría delimitar con mayor precisión la zona de estudio y optimizar tanto la descarga de datos como el rendimiento general del sistema.

Por otra parte, se identifica como una línea de mejora el desarrollo de una versión extendida del modelo actual que contemple múltiples vehículos, lo cual permitiría abordar el Problema de Ruteo de Vehículos (VRP) y sus diversas variantes. Este tipo de extensión es compatible con las capacidades que ofrece `OR-Tools` y permitiría resolver instancias más cercanas a contextos logísticos reales. En la misma línea, la integración de datos de tráfico en tiempo real, utilizando APIs externas como `Google Maps` o `Here`, permitiría adaptar las rutas generadas a condiciones dinámicas del entorno urbano, mejorando la precisión y aplicabilidad del sistema en escenarios prácticos.

Como punto adicional se debe tomar en cuenta que para poder reproducir el código, se debe dar uso a alguna versión anterior de la 2.0 de la librería Osmnx, dado el cambio de los bbox.

Referencias

[Boeing, 2025] Boeing, G. (2025). Modelado y análisis de redes urbanas y servicios con osmnx. *Geographical Analysis*. Publicado en línea antes de su impresión.

[Castro, 2022] Castro, R. D. (2022). Diseño e implementación de una aplicación para la optimización de rutas de entrega mediante el algoritmo de colonia de hormigas. Tesis de licenciatura, Universidad Nacional Mayor de San Marcos. Recuperado de `https://cybertesis.unmsm.edu.pe/item/6cb3df11-13ba-4139-b04c-bacf8ad2083c`.

[Developers, 2024] Developers, F. (2024). Folium documentation — polyline. `https://python-visualization.github.io/folium/latest/user_guide/vector_layers/polyline.html`. Accedido el 30 de junio de 2025.

[Google, 2023] Google (2023). Or-tools: Operations research tools. Google Developers. Recuperado de `https://developers.google.com/optimization`.

[Google OR-Tools Team, 2024] Google OR-Tools Team (2024). *OR-Tools Routing — PathCheapestArc First-Solution Strategy*. Google for Developers. Versión actual consultada en línea.

[Hagberg et al., 2008] Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). Exploring network structure using networkx. *Proceedings of the 7th Python in Science Conference (SciPy)*.

[Muñoz et al., 2024] Muñoz, Salazar, Romero, Gallardo, and Cabrera (2024). Planificación de rutas en mipymes integrando el algoritmo del agente viajero con google maps, ia y python: Un enfoque práctico. *Revista Arandu*. Recuperado de `http://www.uticvirtual.edu.py/revista.ojs/index.php/revistas/article/view/490/727`.